

5장 토큰화

자연어 처리 (Natural Language Processing, NLP)

- 자연어 (Natural Language) : 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어
- 자연어 처리 (NLP) : 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술



자연어 처리의 해결 과제

1. **모호성(Ambiguity)** : 같은 단어와 구가 사용되는 맥락에 따라 다른 의미를 갖게 되므로 이러한 의미를 이해하고 명확하게 구분할 수 있어야 함.
2. **가변성 (Variability)** : 다양한 사투리, 강세, 신조어, 작문 스타일로 인해 매우 가변적인 인간의 언어를 모두 처리하고 이해할 수 있어야 함
3. **구조 (Structure)** : 인간의 언어는 문장이나 구의 의미를 이해할 때 구문을 파악하여 의미를 해석하는데, 알고리즘 역시 문장의 구조와 문법적 요소를 이해하여 의미를 추론하거나 분석할 수 있어야 한다.

⇒ **토큰화(Tokenization)** : 말뭉치(Corpus) 를 컴퓨터가 이해할 수 있는 일정한 단위인 토큰(Token) 으로 나누는 행위

- 토큰나이저 (Tokenizer) : 텍스트 문자열을 토큰으로 나누는 알고리즘 또는 소프트웨어
 - ★ 토큰을 나누는 기준은 구축하려는 시스템이나 주어진 상황에 따라 다른데, 어떻게 토큰을 나누었냐에 따라 시스템의 성능이나 처리 결과가 크게 달라진다.
 - 공백 분할
 - 정규표현식 사용
 - 어휘 사전 (Vocabulary) 사용
 - 머신러닝 사용

단어 및 글자 토큰화

단어 토큰화 (Word Tokenization)

텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업

- 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 토큰화가 수행
 - split 메서드를 이용한 토큰화
주어진 구분자를 통해 문자열을 리스트 데이터로 나누고, 구분자를 입력하지 않으면 공백을 기준으로 나눈다.

#단어 토큰화

```
review = "현실과 구분 불가능한 cg, 시각적 즐거움은 최고! 더불어 ost는 더더욱  
tokenized = review.split()  
print(tokenized)
```

🔗 ['현실과', '구분', '불가능한', 'cg,', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고!!']

글자 토큰화 (Character Tokenization)

띄어쓰기뿐만 아니라 글자 단위로 문장을 나누는 방식으로, 비교적 작은 단어 사전을 구축할 수 있음.

#글자 토큰화

```
review = "현실과 구분 불가능한 cg, 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = list(review)  
print(tokenized)
```

🔗 ['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', ',', '시', '각', '적', ' ', '즐', '거', '움', '은', ' ', '최', '고', '!', ' ', '더', '불', '여', ' ', 'o', 's', 't', '는', ' ', '더', '더', '욱', ' ', '최', '고', '!!']

★ 한글의 경우 영어와 달리 하나의 글자가 여러 자음과 모음의 조합으로 되어 있으므로 자소(언어의 문자 체계에서 의미상 구별할 수 있는 가장 작은 단위) 단위로 토큰화를 수행

★ 자모(jamo) 라이브러리 사용

- 자모 변환 함수 : 입력된 한글 문자열(완성형) 을 유니코드 U+1100 ~ U+11FE 사이의 조합형 한글 자모로 변환

```
retval = jamo.h2j(  
    hangul_string  
)
```

- ## 한글 호환성 자모 변환 함수

```
retval = jamo.j2hcj(
    jamo
)
```

```
#자소 단위 토큰화
#자모 변환 함수
from jamo import h2j, j2hcj

decomposed = j2hcj(h2j(review))
tokenized = list(decomposed)
print(tokenized)
```

→ ['ㅎ', 'ㅌ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅊ', 'ㅣ', 'ㄱ', 'ㅌ', 'ㅁ', 'ㅌ', 'ㄴ', 'ㅣ', 'ㅁ', 'ㅌ', 'ㄹ', 'ㄱ', 'ㅌ']

형태소 토큰화(Morpheme Tokenization) : 텍스트를 형태소 단위로 나누는 토큰화 방법, 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단어로 분류한다.



형태소 (Morpheme) : 실제로 의미를 가지고 있는 최소의 단위

| '그는 나에게 인사를 했다'

- 자립 형태소 (Free Morpheme) : 스스로 의미를 가지고 있는 형태소
ex) "그", "나", "인사"
- 의존 형태소 (Bond Morpheme) : 다른 형태소와 조합되어 사용되는 형태소
ex) "-는", "-에게", "-를", "했-", "-다"

형태소 어휘 사전 (Morpheme Vocabulary)

자연어 처리에서 사용되는 단어의 집합인 어휘 사전 중에서도 각 단어의 형태소 정보를 포함하는 사전

- 품사 태깅 (POS Tagging) : 텍스트 데이터를 형태소 분석하여 각 형태소에 해당하는 품사(Part of Speech, POS)를 태깅하는 작업

KoNLPy 라이브러리

한국어 자연어 처리를 위해 개발된 라이브러리로, 명사 추출, 형태소 분석, 품사 태깅 등의 기능을 제공. 텍스트 데이터를 전처리하고 분석하기 위한 다양한 도구와 함수를 제공해 텍스트 마이닝, 감성 분석, 토픽 모델링 등 다양한 NLP 작업에 사용된다.

- Okt : 문장을 입력받아 명사, 구 형태소, 품사 등의 정보를 추출하는 여러 메서드 제공

표 5.1 Okt 품사 태그

태그	품사	태그	품사
Noun	명사	Punctuation	구두점
Verb	동사	Foreign	외국어
Adjective	형용사	Alpha	알파벳
Determiner	관형사	Number	숫자
Adverb	부사	KoreanParticle	자음/모음
Conjunction	접속사	URL	웹 주소
Exclamation	감탄사	Hashtag	해시태그(#)
Josa	조사	Email	이메일
PreEomi	선어말어미	ScreenName	아이디 표기(@)
Eomi	어미	Unknown	미등록
Suffix	접미사		

- 꼬꼬마 : Kkma 클래스를 통해 명사, 문장, 형태소, 품사를 추출
 - 구문 추출 기능은 지원하지 않지만 명사 추출 기능을 제공.
 - Okt (19개) 에 비해 훨씬 많은 품사 (56개) 를 태깅.

표 5.2 꼬꼬마 품사 태그

태그	품사	태그	품사
NNG	보통명사	EPH	존칭 선어말 어미
NNP	고유명사	EPT	시제 선어말 어미
NNB	일반 의존 명사	EPP	공손 선어말 어미
NNM	단위 의존 명사	EFN	평서형 종결 어미
NR	수사	EFQ	의문형 종결 어미
NP	대명사	EF0	명령형 종결 어미
VV	동사	EFA	청유형 종결 어미
VA	형용사	EFI	감탄형 종결 어미
VXV	보조 동사	EFR	존칭형 종결 어미
VXA	보조 형용사	ECE	대등 연결 어미
VCP	긍정 지정사	ECS	보조적 연결 어미
VCN	부정 지정사	ECD	의존적 연결 어미
MDN	수 관형사	ETN	명사형 전성 어미
MDT	일반 관형사	ETD	관형형 전성 어미

태그	품사	태그	품사
MAG	일반 부사	XPN	체언 접두사
MAC	접속 부사	XPV	용언 접두사
JKS	주격 조사	XSN	명사파생 접미사
JKC	보격 조사	XSV	동사 파생 접미사
JKG	관형격 조사	XSA	형용사 파생 접미사
JKO	목적격 조사	XR	어근
JKM	부사격 조사	SF	마침표, 물음표, 느낌표
JKI	호격 조사	SE	줄임표
JKQ	인용격 조사	SS	따옴표, 괄호표, 줄표
JC	접속 조사	SP	쉼표, 가운뎃점, 콜론, 빗금
JX	보조사	S0	불임표(물결, 숨김, 빠짐)
OH	한자	SW	기타기호(논리수학기호, 화폐기호)
OL	외국어	IC	감탄사
ON	숫자	UN	명사추정범주

• Okt 와 꼬꼬마 를 사용한 형태소 분석 비교

· ['무엇', '상상', '수', '사람', '무엇', '널', '수']
 ['무엇', '상상', '상상할 수', '상상할 수 있는 사람', '사람']
 [('무엇', 'Noun'), ('이트', 'Josa'), ('상상', 'Noun'), ('할', 'Verb'), ('수', 'Noun'), ('있는', 'Adjective'), ('무엇', '이트', '상상', '할', '수', '있는', '사람', '은', '무엇', '이트', '만들어', '널', '수', '있다', '.']

Okt

· ['무엇', '상상', '수', '사람', '무엇']
 ['무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다.']
 [('무엇', 'NNG'), ('이', 'VCP'), ('든', 'ECE'), ('상상', 'NNG'), ('하', 'XSV'), ('=', 'ETD'), ('수', 'NNB'), ('있', 'VV'), ('무엇', '이', '든', '상상', '하', '=', '수', '있', '는', '사람', '은', '무엇', '이', '든', '만들', '어', '내', '=', '수']

꼬꼬마

NLTK (Natural Language Toolkit) 라이브러리

자연어 처리를 위해 개발된 라이브러리로, 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등과 같은 기능을 제공 (영어 기반, 네덜란드/프랑스/독일 등 다양한 언어 지원)

- Treebank 라는 대규모의 영어 말뭉치를 기반으로 학습
- Punkt 모델
- Averaged Perceptron Tagger 모델

표 5.3 Averaged Perceptron Tagger 품사 태그

태그	품사	태그	품사
CC	접속사	PDT	관사
CD	기수(Cardinal Number)	POS	소유격 조사
DT	관형사	PRP	인칭 대명사
EX	there의 대명사형태	PRP\$	소유격 대명사
FW	외래어	RB	부사

태그	품사	태그	품사
JN	전치사	RBR	비교급 부사
JJ	형용사	RBS	최상급 부사
JJR	비교급 형용사	RP	전치사 또는 조동사
JJS	최상급 형용사	VB	기본형 동사
LS	목록 표시	VBD	과거형 동사
MD	조동사	VBG	현재 부사형 동사
NN	단수형 명사	VBN	과거 분사형 동사
NNS	복수형 명사	VBP	1인칭 단수 현재형 동사
NNP	단수형 고유 명사와 서수(Ordinal Number)	VBZ	3인칭 단수 현재형 동사
NNPS	복수형 고유 명사	WDT	관계사
SYM	기호	WP	주격 관계대명사
TO	To 부정사	WP\$	소유격 관계대명사
UH	감탄사	WRB	관계부사

#영문 토큰화

```
from nltk import tokenize
```

```
sentence = "Those who can imagine anything, can create the impossible."
```

```
word_tokens = tokenize.word_tokenize(sentence)
```

```
sent_tokens = tokenize.sent_tokenize(sentence)
```

```
print(word_tokens)
```

```
print(sent_tokens)
```



```
[ 'Those', 'who', 'can', 'imagine', 'anything', ',', 'can', 'create', 'the', 'impossible', '.' ]
['Those who can imagine anything, can create the impossible.']
```

spaCy 라이브러리

사이썬 (Cython) 기반으로 개발된 오픈 소스 라이브러리, 자연어 처리를 위한 기능을 제공. NLTK 와 달리 속도가 빠르고 머신러닝을 기반으로 한다.

- 사전 학습된 모델 en_core_web_sm 사용
- spaCy 품사 태그

표 5.4 spaCy 품사 태그

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
ADJ	JJ	형용사	NUM	CD	기수(Cardinal Number)
ADJ	JJR	비교급 형용사	PROPN	NNP	고유 명사
ADJ	JJS	최상급 형용사	PROPN	NNPS	복수형 고유 명사
ADP	IN	전치사	PUNCT	-LRB-	왼쪽 괄호
ADV	RB	부사	PUNCT	-RRB-	오른쪽 괄호
ADV	RBR	비교급 부사	PUNCT	,	쉼표
ADV	RBS	최상급 부사	PUNCT	:	콜론
ADV	WRB	관계 부사	PUNCT	.	마침표, 물음표, 느낌표
AUX	MD	조동사	PUNCT	...	생략 부호
CCONJ	CC	접속사	PUNCT	“	여는 따옴표
DET	DT	관사, 한정사	PUNCT	”	닫는 따옴표
DET	PDT	전치사 한정사	PUNCT	HYPH	하이픈
DET	PRP\$	소유격 대명사	PUNCT	NFP	불필요한 문장 부호
DET	WDT	관계 대명사	SYM	\$	통화
DET	WP\$	소유격 관계 대명사	VERB	VB	동사 기본형
INTJ	UH	감탄사	VERB	VBD	과거형 동사
NOUN	NN	명사	VERB	VBG	동명사, 현재분사
NOUN	NNS	복수형 명사	VERB	VCN	과거분사
PART	POS	소유격 조사	VERB	VBP	동사 현재형
PART	RP	부사적 불변화사	VERB	VBZ	3인칭 단수 동사 현재형

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
PART	T0	To 부정사	X	ADD	이메일
PRON	EX	존재문(there)	X	FW	외래어
PRON	PRP	인칭 대명사	X	LS	목록
PRON	WP	관계 대명사	X	SYM	기호

#spaCy 품사 태깅

```
import spacy
```

```
nlp = spacy.load("en_core_web_sm")
```

```
sentence = "Those who can imagine anything, can create the impossible."
```

```
doc = nlp(sentence)
```

```
for token in doc:
```

```
    print(f"[{token.pos_:5}-{token.tag_:3}] : {token.text}")
```



```
[PRON -DT ] : Those
[PRON -WP ] : who
[AUX -MD ] : can
[VERB -VB ] : imagine
[PRON -NN ] : anything
[PUNCT-, ] : ,
[AUX -MD ] : can
[VERB -VB ] : create
[DET -DT ] : the
[ADJ -JJ ] : impossible
[PUNCT-. ] : .
```

하위 단어 토큰화

현대 자연어 처리에서는 신조어의 발생, 오타자, 축약어 등을 고려해야 하므로 분석할 단어의 양이 많아져 어려움을 겪는다. 이를 해결하기 위한 방법으로 **하위 단어 토큰화**

(Subword Tokenization) 사용

하위 단어 토큰화 (Subword Tokenization) : 하나의 단어가 빈번하게 사용되는 하위 단어 (Subword) 의 조합으로 나누어 토큰화 하는 방법

| 'Reinforcement' → 'Rein', 'force', 'ment'

바이트 페어 인코딩 (Byte Pair Encoding, BPE)

텍스트 데이터에서 가장 빈번하게 등장하는 글자 쌍의 조합을 찾아 부호화 하는 압축 알고리즘.

연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복하며 이 과정에서 자주 등장하는 단어는 하나의 토큰으로 토큰화 되고, 덜 등장하는 단어는 여러 토큰의 조합으로 표현된다.



예시) 'abracadabra'

1. ab → A
'AracadAra'

2. ra → B
'ABcadAB'

3. AB → c
'CcadC'

- 토큰나이저로써 바이트 페어 인코딩은 자주 등장하는 글자 쌍을 찾아 치환하는 대신 어휘 사전에 추가한다.

- 토큰나이저에서 작동 예시

- 빈도 사전: ('low', 5), ('lower', 2), ('newest', 6), ('widest', 3)
- 어휘 사전: ['low', 'lower', 'newest', 'widest']

- 글자 단위로 나누기

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
- 어휘 사전: ['d', 'e', 'l', 'l', 'n', 'o', 'r', 's', 't', 'w']

- 자주 등장한 글자 쌍 찾기

'e', 's' → 'es'

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'es', 't', 6), ('w', 'i', 'd', 'es', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es']

'es' + 't' → 'est'

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est']

- 결과

- 빈도 사전: ('low', 5), ('low', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est', 'lo', 'low', 'ne', 'new', 'newest', 'wi', 'wid', 'widest']

센텐스피스 라이브러리 / 코포라 라이브러리

- **센텐스피스 라이브러리** : 구글에서 개발한 오픈소스 라이브러리로, 바이트 페어 인코딩과 유사한 알고리즘을 사용해 입력 데이터를 토큰화하고 단어 사전 생성
- **코포라 라이브러리** : 국립국어원이나 AI hub 에서 제공하는 말뭉치 데이터를 쉽게 사용할 수 있게 제공하는 오픈소스 라이브러리.
- 청와대 청원 데이터 실습

```
#센텐스피스-바이트 페어 인코딩 토크나이저 모델 학습
```

```
from Korpora import Korpora
```

```
corpus = Korpora.load("orean_petitions")  
dataset = corpus.train  
petition = dataset[0]
```

```
print("청원 시작일 :", petition.begin)  
print("청원 종료일 :", petition.end)  
print("청원 동의 수 :", petition.num_agree)
```

```
print("청원 범주 :", petition.category)
print("청원 제목 :", petition.title)
print("청원 본문 :", petition.text[:30])
```

청원 시작일 : 2017-08-25
청원 종료일 : 2017-09-24
청원 동의 수 : 88
청원 범주 : 육아/교육
청원 제목 : 학교는 인력센터, 취업센터가 아닙니다. 정말 간곡히 부탁드립니다.
청원 본문 : 안녕하세요. 현재 사대, 교대 등 교원양성학교들의 예비

첫 번째 데이터

◦ 학습 데이터셋 생성

#get_all_texts 메서드로 본문 데이터셋을 한 번에 불러오기

```
corpus = Korpora.load("korean_petitions")
petitions = corpus.get_all_texts()
```

```
with open("corpus.txt", "w", encoding="utf-8") as f:
    for petition in petitions:
        f.write(petition + "\n")
```

◦ 토큰나이저 모델 학습

```
#토큰나이저 모델 학습
from sentencepiece import SentencePieceTrainer

SentencePieceTrainer.Train(
    "--input=corpus.txt\
    --model_prefix=petition_bpe\
    --vocab_size=8000 model_type=bpe"
)
```

⇒ 결과적으로 **petition_bpe.model 파일** (학습된 토큰나이저 저장),
petition_bpe.vocab 파일(어휘 사전 저장) 생성

★ 센텐스피스 라이브러리의 주요한 하이퍼파라미터

표 5.5 SentencePieceTrainer 매개변수

매개변수	의미
input	말뭉치 텍스트 파일의 경로
model_prefix	모델 파일 이름
vocab_size	어휘 사전 크기
character_coverage	말뭉치 내에 존재하는 글자 중 토크나이저가 다룰 수 있는 글자의 비율

model_type	토크나이저 알고리즘 ■ unigram ■ bpe ■ char ■ word
max_sentence_length	최대 문장 길이
unk_id	어휘 사전에 없는 OOV를 의미하는 unk 토큰의 id (기본값: 0)
bos_id	문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값: 1)
eos_id	문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값: 2)

◦ 바이트 페이 인코딩 토큰화

```
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

sentence = "안녕하세요, 토크나이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토크나이저를 사용할 수 있"]

tokenized_sentence = tokenizer.encode_as_pieces(sentence)
tokenized_sentences = tokenizer.encode_as_pieces(sentences)

print("단일 문장 토큰화 :", tokenized_sentence)
print("여러 문장 토큰화 :", tokenized_sentences)

encoded_sentence = tokenizer.encode_as_ids(sentence)
encoded_sentences = tokenizer.encode_as_ids(sentences)

print("단일 문장 인코딩 :", encoded_sentence)
```

```
print("여러 문장 인코딩 :", encoded_sentences)
```

```
decode_ids = tokenizer.decode_ids(encoded_sentences)
decode_pieces = tokenizer.decode_pieces(encoded_sentences)
print("인코딩 값을 문장으로 디코딩 :", decode_ids)
print("인코딩 값을 토큰으로 디코딩 :", decode_pieces)
```

```
☞ 단일 문장 토큰화 : ['__안녕하세요', ' ', '__토', '크', '나', '이', '저', '가', '__잘', '__학', '습', '되었', '군!']
여러 문장 토큰화 : [['__이렇게', '__인', '력', '값을', '__리', '스트', '로', '__받아서'], ['__쉽게', '__토', '크']]
단일 문장 인코딩 : [667, 6553, 994, 6880, 6544, 6513, 6590, 6523, 161, 110, 6554, 872, 787, 6648]
여러 문장 인코딩 : [[372, 182, 6677, 4433, 1772, 1613, 6527, 4162], [1681, 994, 6880, 6544, 6513, 6590, 6536, 58]]
인코딩 값을 문장으로 디코딩 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토큰라이저를 사용할 수 있습니다.']
인코딩 값을 토큰으로 디코딩 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토큰라이저를 사용할 수 있습니다.']
```

◦ 단어 사전 불러오기

```
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

vocab = {idx: tokenizer.id_to_piece(idx) for idx in range(tokenizer.get_piece_size())}

print(list(vocab.items())[:5])
print("어휘 사전 크기 :", len(vocab))
```

```
☞ [(0, '<unk>'), (1, '<s>'), (2, '</s>'), (3, '니다'), (4, '__이')]
어휘 사전 크기 : 8000
```

get_piece_size 메서드 - 센텐스피스 모델에서 생성된 하위 단어의 개수 반환

id_to_piece 메서드 - 정수값을 하위 단어로 변환

<unk> 토큰 - OOV 발생 시 매핑

<s>, </s> - 문장의 시작 지점과 종료 지점을 표시

워드피스 알고리즘

토큰라이저와 유사한 방법으로 학습되지만, 빈도 기반이 아닌 확률 기반으로 글자 쌍을 병합. 모델이 새로운 하위 단어를 생성할 때 이전 하위 단어와 함께 나타날 확률을 계산해 가장 높은 확률을 가진 하위 단어를 선택한다.

수식 5.1 글자 쌍 병합 점수 수식

$$score = \frac{f(x,y)}{f(x),f(y)}$$

f : 빈도(frequency) 를 나타내는 함수 / x,y : 병합하려는 하위 단어.

- 워드피스 알고리즘 작동 예시

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

- 병합할 단어 쌍 선정

'e' + 's' → 0.06 / 'i' + 'd' → 0.33 ⇒ 'i' 와 'd' 병합

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'id', 'e', 's', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'id']

'l' + 'o' → 0.14 ⇒ 'l' 와 'o' 병합

- 빈도 사전: ('lo', 'w', 5), ('lo', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'id', 'e', 's', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'id', 'lo']

토크나이저스 라이브러리

- 정규화 (Normalization) : 일관된 형식으로 텍스트를 표준화하고 모호한 경우를 방지하기 위해 일부 문자를 대체하거나 제거
ex) 불필요한 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수 문자 처리 등

- 사전 토큰화 (Pre-Tokenization) : 입력 문장을 토큰화하기 전에 단어와 같은 작은 단위로 나누는 기능.
- 실습
 - 워드피스 토크나이저 학습

```
from tokenizers import Tokenizer
from tokenizers.models import WordPiece
from tokenizers.normalizers import Sequence, NFD, Lowercase
from tokenizers.pre_tokenizers import Whitespace
from tokenizers.trainers import WordPieceTrainer

tokenizer = Tokenizer(WordPiece())
tokenizer.normalizer = Sequence([NFD(), Lowercase()]) #정규화 방식 지정
tokenizer.pre_tokenizer = Whitespace() #사전 토큰화 방식 지정

tokenizer.train(["corpus.txt"])
tokenizer.save("tokenizer.json")
```

- 워드피스 토큰화

```
from tokenizers import Tokenizer
from tokenizers.decoders import WordPiece as WordPieceDecoder

#모델 결과가 저장된 json 파일을 불러와 Tokenizer 객체 설정, WordPieceDecoder
tokenizer = tokenizer.from_file("tokenizer.json")
tokenizer.decoder = WordPieceDecoder()

sentence = "안녕하세요, 토크나이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토크나이저를 사용할 수 있"]

#encoding 메서드로 문장을 토큰화
encoded_sentence = tokenizer.encode(sentence)
#encoding_batch 메서드로 여러 문장을 한 번에 토큰화
encoded_sentences = tokenizer.encode_batch(sentences)

print("인코더 형식: ", type(encoded_sentence))
```

```

print("단일 문장 토큰화 :", encoded_sentence.token)
print("여러 문장 토큰화 :", [enc.tokens for enc in encoded_sentences])

print("단일 문장 정수 인코딩 :", encoded_sentence.ids)
print("여러 문장 정수 인코딩 :", [enc.ids for enc in encoded_sentences])

print("정수 인코딩에서 문장 변환 :", tokenizer.decode(encoded_sentence.ids))

```

인코더 형식 : <class 'tokenizers.Encoding'>

단일 문장 토큰화 : ['안녕하세요', '.', '위', '##드', '##피', '##스', '토', '##크', '##나이', '##저', '##입니다', '.']

여러 문장 토큰화 : [['입력', '##값을', '리스트', '##로', '전달', '##해'], ['쉽게', '토', '##크', '##나이', '##저', '##를', '사용할', '수', '있습니다', '.']]

단일 문장 정수 인코딩 : [8760, 13, 10398, 7638, 7955, 7512, 8693, 8415, 16269, 7536, 7598, 13]

여러 문장 정수 인코딩 : [[19643, 13834, 28119, 7495, 11589, 7502], [9739, 8693, 8415, 16269, 7536, 7510, 14129, 7562, 7698]]

정수 인코딩에서 문장 변환 : 안녕하세요. 워드피스 토크나이저입니다.